

Lecture 3
Lambdas, Stream API, Executors, CompletableFutures, Virtual Threads

S.l. Dr. Ing. Radu MIRON
radu.miron@aut.utcluj.ro

Lambda Expressions

DOCUMENTATION:
https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html



Lambda Expressions

Lambda expressions are *syntactic sugar* for implementing functional interfaces.

Functional interfaces are interfaces that have a single abstract method (without implementation).

They represent a much more concise alternative to using anonymous classes.

They form the basis of functional programming in Java.

Introduced in Java 8 (2014).

Exemples:

() -> System.out.println("Something")
x -> x + 2
s -> s.contains("abc")
(a, b) -> a+b



Lambda Expressions

```
@FunctionalInterface
interface Performer {
   int performOperation(String param);
}


public class Main {
   public static void main(String[] args) {

      // anonymous implementation for Performer
      Performer implementation1 = new Performer() {
         @Override
         public int performOperation(String param) {
            return param.compareTo("a");
         }
      };

      System.out.println(
               implementation1.performOperation("b"));// 1 (because "b" > "a")


      // lambda expression implementation for Performer
      Performer implementation2 = param -> param.length();
      System.out.println(
               implementation2.performOperation("abdefg"));// 6
   }
}
```

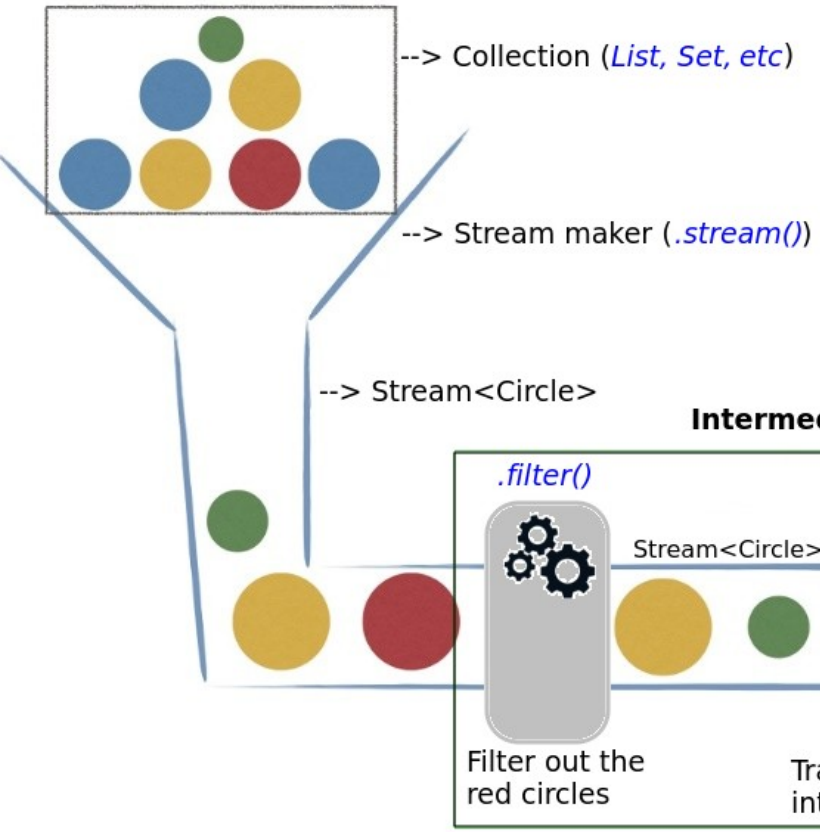
Stream API

DOCUMENTATION:
https://www.digitalocean.com/community/tutorials/java-8-stream
https://www.baeldung.com/java-8-streams
https://stackify.com/streams-guide-java-8/

The Java Stream API is a powerful feature introduced in Java 8, primarily aimed at simplifying and optimizing processing sequences of elements in collections. It enables functional-style operations on streams of elements, providing a fluent and concise way to perform common data manipulation tasks.

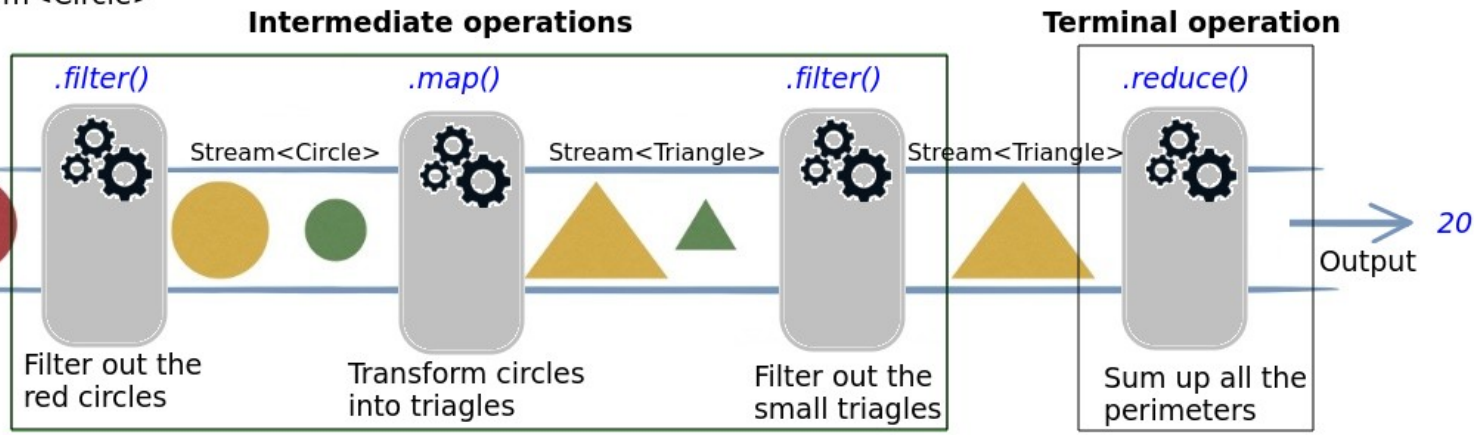
Stream API






```
List<Circle> circles = ...;

int sum = circles.stream()
                 .filter(c -> c.getColor()!="red")
                 .map(c -> c.toTriangle())
                 .filter(t -> t.getArea() > 5)
                 .reduce(0, (t1, t2) ->
                         t1.getPerimeter() + t2.getPerimeter());
```





Stream API

Creating a stream

```
int[] array = {1, 2, 3, 4};
IntStream intStream = Arrays.stream(array);

Stream<String> stringStream1 = Stream.of("abc", "efg", "hij");

List<Character> list = ...;
Stream<Character> characterStream = list.stream();
Stream<String> stringStream =
        List.of("abc", "efg", "hij").stream();

Stream<LocalDate> localDateStream =
        Set.of(LocalDate.now(), LocalDate.of(2021, 1, 1)).stream()

Queue<Boolean> queue = ...;
Stream<Boolean> booleanStream = queue.stream();

Map<String, Integer> map = ...;
Stream<Map.Entry<String, Integer>> entryStream =
        map.entrySet().stream();
```



Stream API

Some Intermediate operations

Stream operation	Goal	Input
filter()	Filters items according to predicate	Predicate
map()	Transforms items	Function
limit()	Limit the items number	int
sorted()	Sorts based on the comparator	Comparator
distinct()	Removes duplicates	-



Stream API

Some terminal operations

Stream operation	Goal	Input
forEach()	Iterates the items	Consumer
count()	Counts the items	-
collect()	Collects the items	int
reduce()	Aggregates the items	BinaryOperator
min(), max()	Return min, max	Comparator
anyMatch(), allMatch(), noneMatch()	Return boolean	Predicate



Stream API

Optional

Java Optional is a container object which may or may not contain a non-null value. It was introduced in Java 8 to address the problem of *NullPointerExceptions* that can occur when attempting to access methods of an object that is null.



Stream API

Optional

```
Optional<String> optional =
   List.of("abc", "efg" /*etc*/).stream()
                                .filter(s -> s.length() > 4)
                                .findAny(); // or . findFirst()


// further map and/or filter an optional
Optional<Integer> optionalInt = optional.map(s -> s.hashCode());
Optional<Integer> optionalInt1 = optionalInt.filter(i -> i % 2 == 0);

// if optional is empty provide an alternative
String s1 = optional.orElse(":|");
String s2 = optional.orElseGet(() -> ":)");
String s3 = optional.orElseThrow(
            () -> new RuntimeException("String not found"));

// if optional is present do something with it
optional.ifPresent(s -> System.out.println(s.length()));
optional.ifPresentOrElse(System.out::println,
                         () -> System.out.println("String not found"));

// check if it's empty or present
boolean empty = optional.isEmpty();
boolean present = optional.isPresent();
```

Executors and Future Variables

DOCUMENTATION:
https://www.baeldung.com/java-executor-service-tutorial



Executors

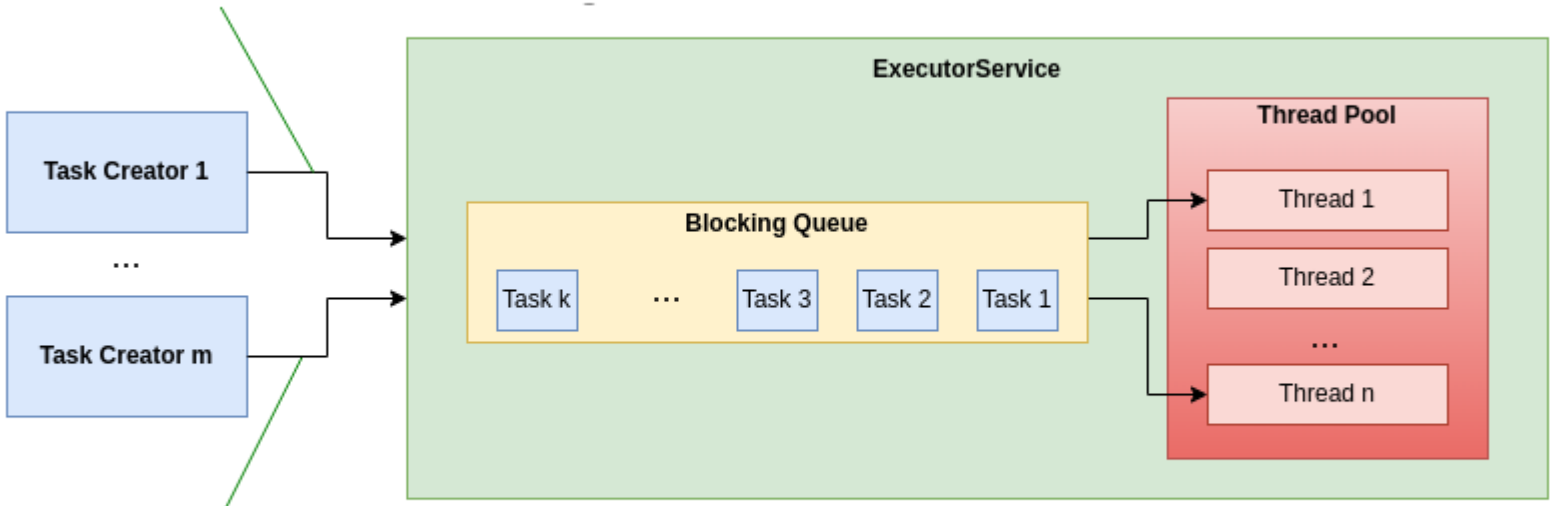
ExecutorService is an interface in Java that provides a framework for executing and managing threads. This interface is part of the **Executor** framework, which offers a more flexible and efficient way to execute tasks in threads in Java applications.

Executors – Future variables

The **Future** interface in Java's **java.util.concurrent** package represents the result of an **asynchronous computation**. It provides a way to check if the computation is completed, retrieve the result, and handle exceptions.

Example that demonstrates the use of **ExecutorService** along with **Future**:

```
executor.execute(new Task()) // fire and forget
 // Task implements Runnable
```




```
ExecutorService executorService = Executors.newFixedThreadPool(n);
Future<Val> fVar= executorService.submit(new Task()); // fork - join
// Task implements Callable
```



Executors – Future variables

```
// Create an ExecutorService with a single thread
ExecutorService executorService =
    Executors.newSingleThreadExecutor();

// Submit a task that returns a Future
Future<String> futureResult =
    executorService.submit(new MyCallableTask());

// Retrieve the result of the task (blocking call)
String result = futureResult.get();


class MyCallableTask implements Callable<String> {
    @Override
    public String call() throws Exception {
        // Simulate some time-consuming task
        Thread.sleep(2000);

        return "Task completed";
    }
}
```

//See course repository for the complete example

CompletableFuture



DOCUMENTATION:
https://www.baeldung.com/java-completablefuture



CompletableFuture

CompletableFuture is a class introduced in Java 8 as part of the **java.util.concurrent** package. It represents a future result of an **asynchronous computation**, providing a rich set of methods for working with asynchronous operations and **chaining multiple asynchronous tasks** together.

CompletableFuture allows you to create **a pipeline of operations that execute asynchronously** and handle their results when they become available. It supports composition, combining, and chaining of asynchronous tasks, enabling developers to write complex asynchronous code in a more readable and maintainable way.



CompletableFuture

```
CompletableFuture.supplyAsync(() -> new Random().nextInt(100))
        .thenApply(r -> r * 3)
        .thenApply(r -> "<" + r + ">")
        .thenAccept(System.out::println)
        .join();
```

supplyAsync(Supplier<T> supplier) - Executes a task in an asynchronous manner and provides a CompletableFuture that represents the eventual result.

thenApply(Function<? super T,? extends U> fn) - Applies a function to the result of a preceding task, generating a CompletableFuture that represents the eventual result.

thenAccept(Consumer<? super T> action) – Consumes the eventual result in the CompletableFuture. It returns a CompletableFuture<Void>.

join() - Returns the result value when complete.

Full method list:
https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/CompletableFuture.html

//See course repository for the complete examples

Virtual Java Threads



DOCUMENTATION:
https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html
https://blog.ycrash.io/apis-to-create-java-virtual-thread/

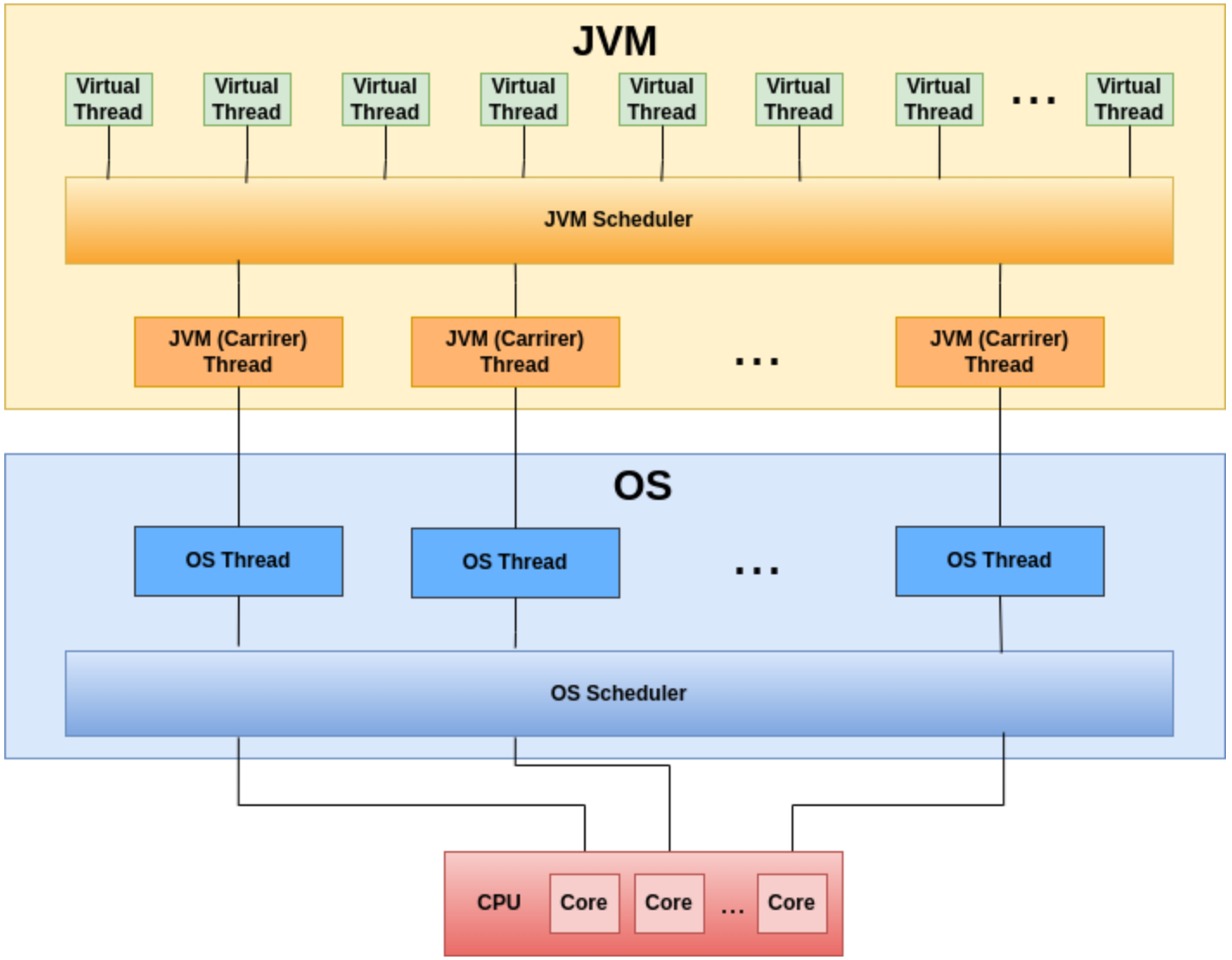


Virtual Threads

In earlier versions of the JVM (Java Virtual Machine), there was only one type of thread: the **“classic”** or **“platform”** thread. When a platform thread is created, it is associated with an operating system thread. This associated operating system thread remains dedicated to the platform thread until the platform thread completes its execution. Afterwards, the operating system thread becomes available for other tasks. Essentially, each platform thread has a direct connection to a specific operating system thread. **Platform threads are therefore orchestrated by the operating system scheduler.**

Virtual threads introduce a layer of abstraction between operating system threads and application-level concurrency. In other words, virtual threads can be used to schedule tasks that the Java Virtual Machine (JVM) coordinates, or schedules, and executes on platform threads, which thus become **carrier threads**.

Virtual Threads





Virtual Threads Creation

1. Thread.startVirtualThread()

2. Thread.ofVirtual().start()

3. Thread.ofVirtual().unstarted()

4. Executors.newVirtualThreadPerTaskExecutor()

5. Executors.newThreadPerTaskExecutor()



“Pinned Virtual Threads”

There are some cases in which a blocking operation does not decouple the virtual thread from the carrier thread, thus blocking the carrier thread as well. In such cases, we say that the virtual thread is pinned to the carrier thread. This is not an error, but rather a behavior that limits the scalability of the application.

Fortunately, there are only two such scenarios:

- *synchronized* code blocks;

- native method calls, that is, the use of JNI.

Therefore, when virtual threads are used, it is recommended to use Lock instead of synchronized to implement mutual exclusion.



The end!